

Chapter 1

Functions

This chapter introduces the most important idea in computer programming: functions! To explain why functions are so important, I'll start by explaining one of the problems they solve: name collisions.

1.1 Name Collisions

All scripts run in the same workspace, so if one script changes the value of a variable, all other scripts see the change. With a few simple scripts, that's not a problem, but eventually the interactions between scripts become unmanageable.

For example, the following script computes the sum of the first `n` terms in a geometric sequence, but it also has the *side effect* of assigning values to `A1`, `total`, `i`, and `a`.

```
A1 = 1;
total = 0;
for i=1:10
    a = A1 * 0.5^(i-1);
    total = total + a;
end
ans = total
```

If you were using any of those variable names before calling this script, you might be surprised to find, after running the script, that their values had changed. If you have two scripts that use the same variable names, you might find that they work separately and then break when you try to combine them. This kind of interaction is called a *name collision*.

As the number of scripts you write increases, and they get longer and more complex, name collisions become more of a problem. One of the several motivations for functions is to avoid this problem by limiting the *scope* of the variables within the function to be *local* to that function. This is one instance of *encapsulation*, a common strategy to manage program complexity. The function encapsulates the local variables so that they are only accessible within the function.

1.2 Defining Functions

A *function* is like a script, except that each function has its own workspace, so any local variables defined inside a function only exist while the function is running and don't interfere with variables in other workspaces, even if they have the same name. In other words, the scope of the local variables within the function are limited to only the operations the function. Function inputs and outputs are defined carefully to avoid unexpected interactions.

To define a new function, you create an M-file with the name you want and put a function definition in it. For example, to create a function named `myfunc`, create an M-file named `myfunc.m` and put the following definition into it (Listing 1.1):

Listing 1.1: A function definition

```
function res = myfunc(x)
    s = sin(x)
    c = cos(x)
    res = abs(s) + abs(c)
end
```

The first non-comment word of the file has to be `function`, because that's how MATLAB tells the difference between a script and a function file.

A function definition is a compound statement. The first line is called the *signature* of the function; it defines the inputs and outputs of the function. In Listing 1.1 the *input variable* is named `x`. When this function is called, the argument provided by the user will be assigned to `x`.

The *output variable* is named `res`, which is short for *result*. You can call the output variable whatever you want, but as a convention, I like to call it `res`. Usually the last thing a function does is assign a value to the output variable.

Once you've defined a new function, you call it the same way you call built-in MATLAB functions. If you call the function as a statement, MATLAB puts the result into `ans`:

```
>> myfunc(1)

s = 0.84147098480790

c = 0.54030230586814

res = 1.38177329067604

ans = 1.38177329067604
```

But it's more common (and better style) to assign the result to a variable:

```
>> y = myfunc(1)

s = 0.84147098480790
```

```
c = 0.54030230586814  
  
res = 1.38177329067604  
  
y = 1.38177329067604
```

While you're debugging a new function, you might want to display intermediate results like this, but once it's working, you'll want to add semicolons to make it a *silent function*. A silent function computes a result but doesn't display anything (except sometimes warning messages). Most built-in functions are silent.

Each function has its own workspace, which is created when the function starts and destroyed when the function ends. If you try to access (read or write) the variables defined inside a function, you will find that they don't exist.

```
>> clear  
>> y = myfunc(1);  
>> who  
Your variables are: y  
  
>> s  
Undefined function or variable 's'.
```

The only value from the function that you can access is the result, which in this case is assigned to `y`.

If you have variables named `s` or `c` in your workspace before you call `myfunc`, they will still be there when the function completes.

```
>> s = 1;  
>> c = 1;  
>> y = myfunc(1);  
>> s, c  
  
s = 1  
c = 1
```

So inside a function you can use whatever variable names you want without worrying about collisions.

1.3 Function Documentation

At the beginning of every function file, you should include a comment that explains what the function does:

```
function res = myfunc(x)  
%MYFUNC Manhattan distance from the origin to a point on the unit circle.
```

```
% d = MYFUNC(d) is the distance from the origin to a point on the unit
% circle with (x) as the angle in radians.

s = sin(x);
c = cos(x);
res = abs(s) + abs(c);

end
```

When you ask for `help`, MATLAB prints the comment you provide.

```
>> help myfunc
myfunc Manhattan distance from the origin to a point on the unit circle.
d = myfunc(d) is the distance from the origin to a point on the unit
circle with (x) as the angle in radians.
```

There are lots of conventions about what should be included in these comments. Looking at examples of documentation for MATLAB's included functions can be a productive way to understand the style and convention used in documenting functions. You can open code a MATLAB function in an Editor window (read-only) to better see the raw text. For example,

```
>> edit mean
```

will open the *mean.m* function file.

Among other things, it's a good idea to include the following:

Signature The signature of the function, which includes the name of the function, the input variable(s), and the output variable(s).

Description A clear, concise, abstract description of what the function does. An *abstract* description is one that leaves out the details of *how* the function works and includes only information that someone using the function needs to know. You can put additional comments inside the function that explain the details.

Variables An explanation of what the input and output variables mean; for example, in this case it is important to note that `x` is considered to be an angle in radians.

Examples Working examples of how you would use the function. Often functions can be called in different ways, so providing examples of the applicable use cases helps convey the intended implementations.

See Also If there are related functions, listing them can be really helpful for future users, including yourself. For example the `mean` function lists `median`, `median`, `std`, `min`, `max`, `var`, `cov`, all related reduce operations that produce summary statistics.

1.4 Naming Functions

There are a few “gotchas” that come up when you start defining functions. The first is that the “real” name of your function is determined by the file name, *not* by the name you put in the function signature. As a matter of style, you should make sure that they are always the same, but if you make a mistake, or if you change the name of a function, it’s easy to get confused.

In the spirit of making errors on purpose, edit *myfunc.m* and change the name of the function from `myfunc` to `something_else`, like this:

```
function res = something_else (x)
    s = sin(x);
    c = cos(x);
    res = abs(s) + abs(c);
end
```

Now call the function from the Command Window, like this:

```
>> y = myfunc(1)
y = 1.3818
```

The function is still called `myfunc`, because that’s the name of the file. If you try to call it like this:

```
>> y = something_else(1)
Undefined function or variable 'something_else'.
```

It doesn’t work. The name of the file is what matters; the name of the function is ignored.

The second “gotcha” is that the name of the file can’t have spaces. For example, if you rename the file to *my func.m* and try to run it, you get:

```
>> y = my func(1)
y = my func(1)
|
Error: Unexpected MATLAB expression.
```

This fails because MATLAB thinks `my` and `func` are two different variable names.

The third “gotcha” is that your function names can collide with built-in MATLAB functions. For example, if you create an M-file named *sum.m* and then call `sum`, MATLAB might call your new function, not the built-in version! Which one actually gets called depends on the order of the directories in the search path and (in some cases) on the arguments. As an example, put the following code in a file named *sum.m*:

```
function res = sum(x)
    res = 7;
end
```

And then try this:

```
>> sum(1:3)
```

```
ans = 6
```

```
>> sum
```

```
ans = 7
```

In the first case MATLAB used the built-in function; in the second case it ran your function! This kind of interaction can be very confusing. Before you create a new function, check to see if there is already a MATLAB function with the same name. If there is, choose another name!

1.5 Multiple Input Variables

Functions can take more than one input variable. For example, the following function in Listing 1.2 takes two input variables, *a* and *b*:

Listing 1.2: A function that computes the sum of squares of two numbers

```
function res = sum_squares(a, b)
    res = a^2 + b^2;
end
```

This function computes the sum of squares of two numbers, *a* and *b*.

If we call it from the Command Window with arguments 3 and 4, we can confirm that the sum of their squares is 25.

```
>> ss = sum_squares(3, 4)
ss = 25
```

The arguments you provide are assigned to the input variables in order, so in this case 3 is assigned to *a* and 4 is assigned to *b*. MATLAB checks that you provide the right number of arguments; if you provide too few, you get

```
>> ss = sum_squares(3)
Not enough input arguments.
```

```
Error in sum_squares (line 4)
    res = a^2 + b^2;
```

This error message might be confusing, because it suggests that the problem is in `sum_squares` rather than in the function call. Keep that in mind when you're debugging.

If you provide too many arguments, you get

```
ss = sum_squares(3, 4, 5)
Error using sum_squares
Too many input arguments.
```

That's a better error message, because it's clear that the problem isn't in the function, it's in the way we're using the function.

1.6 Multiple Output Variables

Functions can also return multiple output variables. Here is an example of a function that returns two variables:

```
function [s, p] = sum_and_product(a, b)
    s = a + b;
    p = a * b;
end
```

We can call this function and assign the output to variables in the workspace, making sure to include the square brackets, like this:

```
>> [added, multiplied] = sum_and_product(4, 5)
added =
     9
multiplied =
    20
```

There are some potential gotchas here as well. If we only provide one output in the function call, MATLAB will return just the first output variable without a warning or error:

```
>> added = sum_and_product(4, 5)
added =
     9
```

Also, if we don't include the square brackets, both variables will be assigned the first output, which is probably not what we would intend:

```
>> added, multiplied = sum_and_product(4, 5)
added =
     9
multiplied =
     9
```

1.7 Local Functions

The primary way to create a MATLAB function is to have a single function in a single m-file where the function and the file share the same name. An alternative is to have a function within a script or to have multiple functions within the same m-file, called a *local function*. This can be helpful if you don't want to create an manage many separate files. These local functions are only accessible from file in which they are defined; they can't be accessed directly from the command line.

For example, we could create a script named *local_function_demo.m* with the following code:

```
function y = my_local_func(x)
% Local function to compute the square of x.
y = x^2;
end

% Call the local function
b = my_local_func(2)
```

Then we can run the script from the command window:

```
>> local_function_demo
b =
    4
```

which works as expected. You might also notice that you can't call the `my_local_func` directly from the command window, because it is local to the script:

```
>> my_local_function(3)
Unrecognized function or variable 'my_local_function'.
```

This behavior is particularly useful when using live scripts so that you can have various functions and the code that calls them all in the same live script document.

Note: Before R2024a, local functions in scripts must be defined at the end of the file, after the last line of script code.

1.8 Chapter Review

Now that we know about functions, and all the ways they can go wrong, let's put them to good use. In the next chapter we'll develop a program that uses several functions to search for Pythagorean triples (and I'll explain what those are).

Here are a few terms in this chapter you might want to remember.

A *function* is a named sequence of statements stored in an M-file. A function can have one or more *input variables*, which get their values when the function is called, and *output variables*, which return a value from the function to the caller.

The first line of a function definition is its *signature*, which specifies the name of the function, the input variables, and the output variables.

A *silent function* doesn't display anything, generate a figure, or have any other effect other than returning output values.

1.9 Exercise

Before you go on, you might want to work on the following exercise.

Exercise 1.1. Write a function called `hypotenuse` that takes two parameters, `a` and `b`, that represent the lengths of two sides of a right triangle. It should assign to `res` the length of the third side of the triangle, given by the formula

$$c = \sqrt{a^2 + b^2}$$